

Data Analytics (DA)

Module-End -4

Module End Assignment: PYTHON

“Customer Support Ticket Analyzer”

Project Title: “Customer Support Ticket Analyzer”

Problem Statement

Customer support teams handle numerous service tickets daily. Analysing support tickets helps identify common issues, customer sentiment, support quality, and areas for improvement.

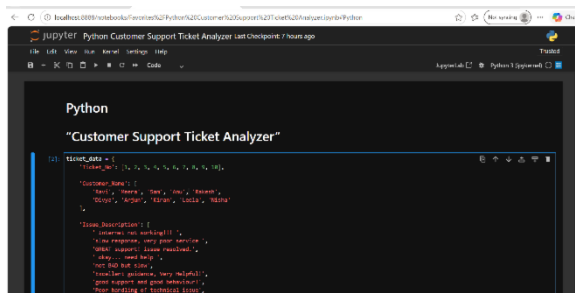
In this project, you will build a **Python-based Ticket Analysis System that stores, cleans, analyses, and extracts insights from customer support tickets.**

Step 1: Preloaded Tickets

Begin your program with the following dictionary of lists containing 10 customer tickets:

```
ticket_data = { 'Ticket_No': [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],
'Customer_Name': [ 'Ravi', 'Meera', 'Sam', 'Anu', 'Rakesh', 'Divya', 'Arjun', 'Kiran', 'Leela',
'Nisha'
],
'Issue_Description': [
'Internet not working!!! ', 'slow response, very poor service ',
'GREAT support! issue resolved.', 'okay... need help ', 'not BAD but slow', 'Excellent guidance,
Very Helpful!',
'good support and good behaviour!', 'Poor handling of technical issue', 'Satisfied. Could be
better.', 'Good service...
quick response.'
],
'Priority': ['High', 'Low', 'High', 'Medium', 'Low', 'High', 'Medium', 'High', 'Low', 'Medium']}
print(ticket_data)
```

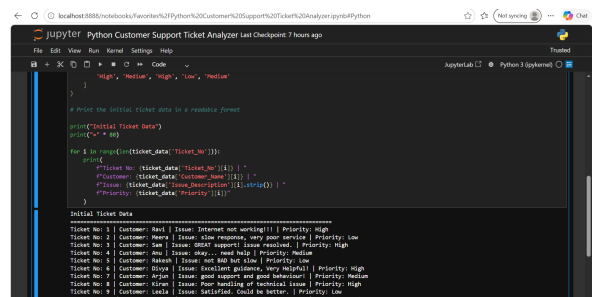
OUTPUT:



```
Python
"Customer Support Ticket Analyzer"

ticket_data = {
    'Ticket_No': [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],
    'Customer_Name': [
        'Ravi', 'Meera', 'Sam', 'Anu', 'Rakesh', 'Divya',
        'Arjun', 'Kiran', 'Leela', 'Nisha'
    ],
    'Issue_Description': [
        'Internet not working!!! ', 'slow response, very poor service ',
        'GREAT support! issue resolved.', 'okay... need help ',
        'not BAD but slow', 'Excellent guidance, Very Helpful!',
        'good support and good behaviour!', 'Poor handling of technical issue',
        'Satisfied. Could be better.', 'Good service... quick response.'
    ],
    'Priority': ['High', 'Low', 'High', 'Medium', 'Low', 'High', 'Medium', 'High', 'Low', 'Medium']}

print(ticket_data)
```



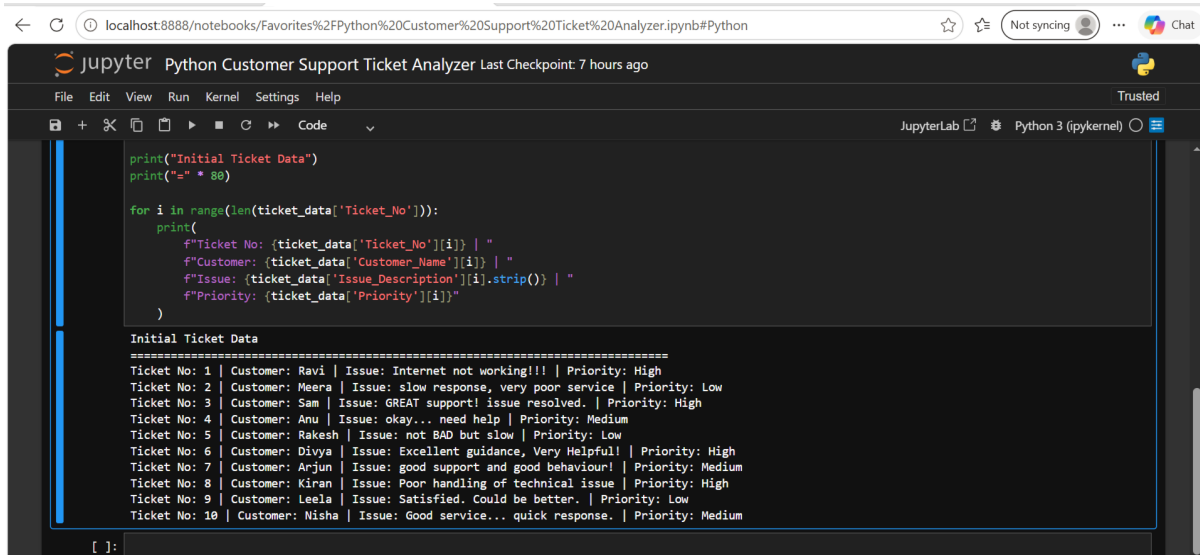
```
Python Customer Support Ticket Analyzer Last Checkpoint: 7 hours ago

# Print the initial ticket data in a readable format
print("\nInitial Ticket Data")
print("-" * 80)

for i in range(len(ticket_data['Ticket_No'])):
    print(f"Ticket No: {ticket_data['Ticket_No'][i]} | "
          f"Customer: {ticket_data['Customer_Name'][i]} | "
          f"Issue: {ticket_data['Issue_Description'][i].strip()} | "
          f"Priority: {ticket_data['Priority'][i]}")

Initial Ticket Data
Ticket No: 1 | Customer: Ravi | Issue: Internet not working!!! | Priority: High
Ticket No: 2 | Customer: Meera | Issue: slow response, very poor service | Priority: Low
Ticket No: 3 | Customer: Sam | Issue: GREAT support! issue resolved. | Priority: High
Ticket No: 4 | Customer: Anu | Issue: okay... need help | Priority: Medium
Ticket No: 5 | Customer: Rakesh | Issue: not BAD but slow | Priority: Low
Ticket No: 6 | Customer: Divya | Issue: Excellent guidance, very helpful | Priority: High
Ticket No: 7 | Customer: Arjun | Issue: good support and good behaviour | Priority: Medium
Ticket No: 8 | Customer: Kiran | Issue: Poor handling of technical issue | Priority: High
Ticket No: 9 | Customer: Leela | Issue: Satisfied. Could be better. | Priority: Low
Ticket No: 10 | Customer: Nisha | Issue: Good service... quick response. | Priority: Medium
```

- Task: Print the initial ticket data in a readable format.



The screenshot shows a JupyterLab window titled "Python Customer Support Ticket Analyzer". The code cell contains a loop that prints ticket details for 10 tickets. The output shows a table-like format with columns for Ticket No., Customer, Issue, and Priority.

```
print("Initial Ticket Data")
print("=" * 80)

for i in range(len(ticket_data['Ticket_No'])):
    print(
        f'Ticket No: {ticket_data['Ticket_No'][i]} | "
        f'Customer: {ticket_data['Customer_Name'][i]} | "
        f'Issue: {ticket_data['Issue_Description'][i].strip()} | "
        f'Priority: {ticket_data['Priority'][i]}"'
    )
```

Initial Ticket Data

Ticket No.	Customer	Issue	Priority
1	Ravi	Internet not working!!!	High
2	Meera	slow response, very poor service	Low
3	Sam	GREAT support! issue resolved.	High
4	Anu	okay... need help	Medium
5	Rakesh	not BAD but slow	Low
6	Divya	Excellent guidance, Very Helpful	High
7	Arjun	good support and good behaviour!	Medium
8	Kiran	Poor handling of technical issue	High
9	Leela	Satisfied. Could be better.	Low
10	Nisha	Good service... quick response.	Medium

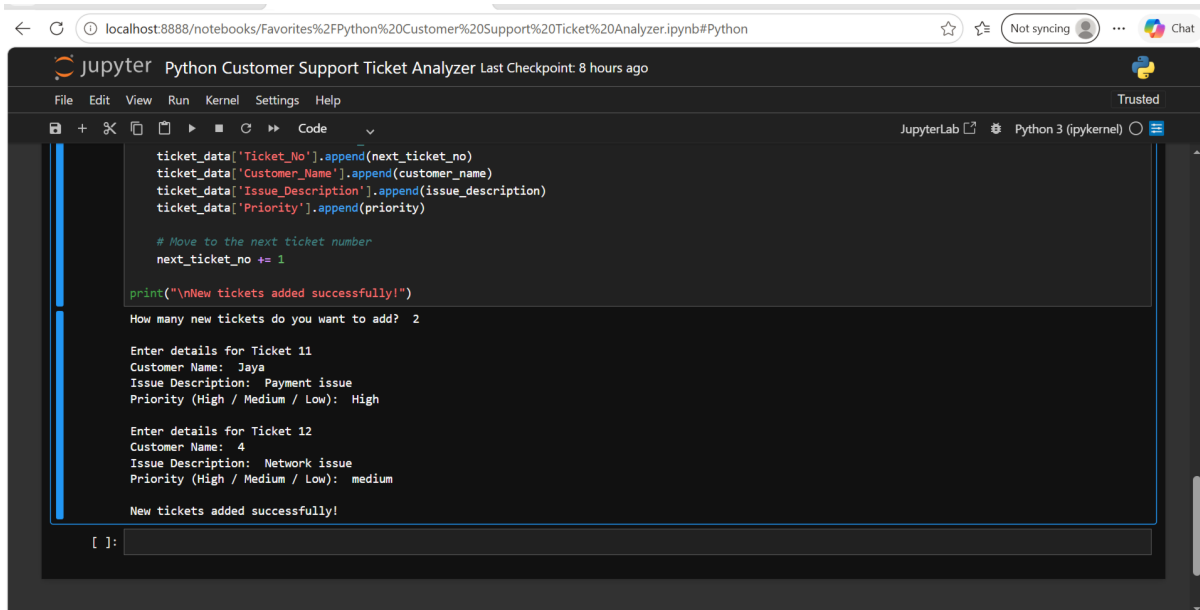
Step 2: Add More Tickets

- ❖ Ask the user: How many new tickets do you want to add?
- ❖ For each new ticket, collect:
 - Customer Name
 - Issue Description
 - Priority (High / Medium / Low)

Requirements:

- Ticket numbers must auto-increment starting from 11.
- Validate priority (accept only High, Medium, Low).
- Append new data to ticket_data.

OUTPUT:



The screenshot shows the JupyterLab window after adding two new tickets. The code cell uses the .append() method to add new data to the ticket_data dictionary. The output shows the user being prompted for the number of tickets to add (2), followed by prompts for each ticket's details (Customer Name, Issue Description, Priority). The final output shows the new tickets added successfully.

```
ticket_data['Ticket_No'].append(next_ticket_no)
ticket_data['Customer_Name'].append(customer_name)
ticket_data['Issue_Description'].append(issue_description)
ticket_data['Priority'].append(priority)

# Move to the next ticket number
next_ticket_no += 1

print("\nNew tickets added successfully!")
```

How many new tickets do you want to add? 2

Enter details for Ticket 11
Customer Name: Jaya
Issue Description: Payment issue
Priority (High / Medium / Low): High

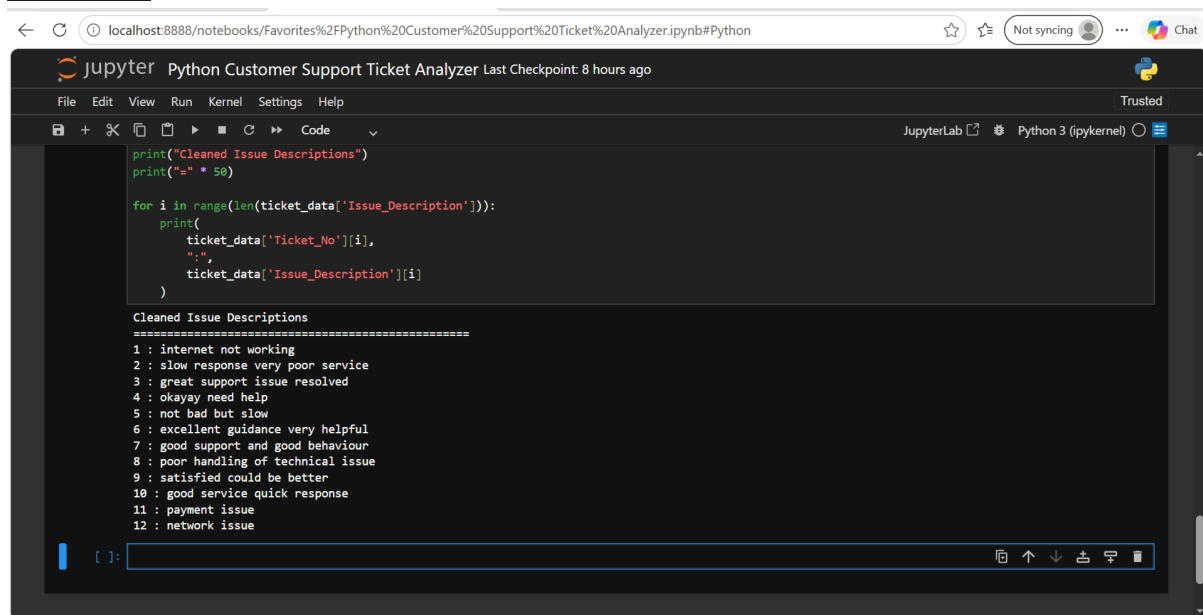
Enter details for Ticket 12
Customer Name: 4
Issue Description: Network issue
Priority (High / Medium / Low): medium

New tickets added successfully!

Step 3: Text Cleaning for Issue Descriptions

- ❖ Clean all issue descriptions by applying:
 - Remove punctuation (.,!?-)
 - Convert multiple spaces → single space
 - Remove leading/trailing spaces
 - Convert text to lowercase
 - Replace slang/shorthand (example: "ok" → "okay")
- ❖ Hints: *Use .replace(), .split(), ' '.join(), .strip(), .lower().*

OUTPUT:



```
print("Cleaned Issue Descriptions")
print("=" * 50)

for i in range(len(ticket_data['Issue_Description'])):
    print(
        ticket_data['Ticket_No'][i],
        ":",
        ticket_data['Issue_Description'][i]
    )

Cleaned Issue Descriptions
=====
1 : internet not working
2 : slow response very poor service
3 : great support issue resolved
4 : okayay need help
5 : not bad but slow
6 : excellent guidance very helpful
7 : good support and good behaviour
8 : poor handling of technical issue
9 : satisfied could be better
10 : good service quick response
11 : payment issue
12 : network issue
```

Step 4: Keyword-Based Issue Insights

Create a function:

def count_tickets_with_word(word):

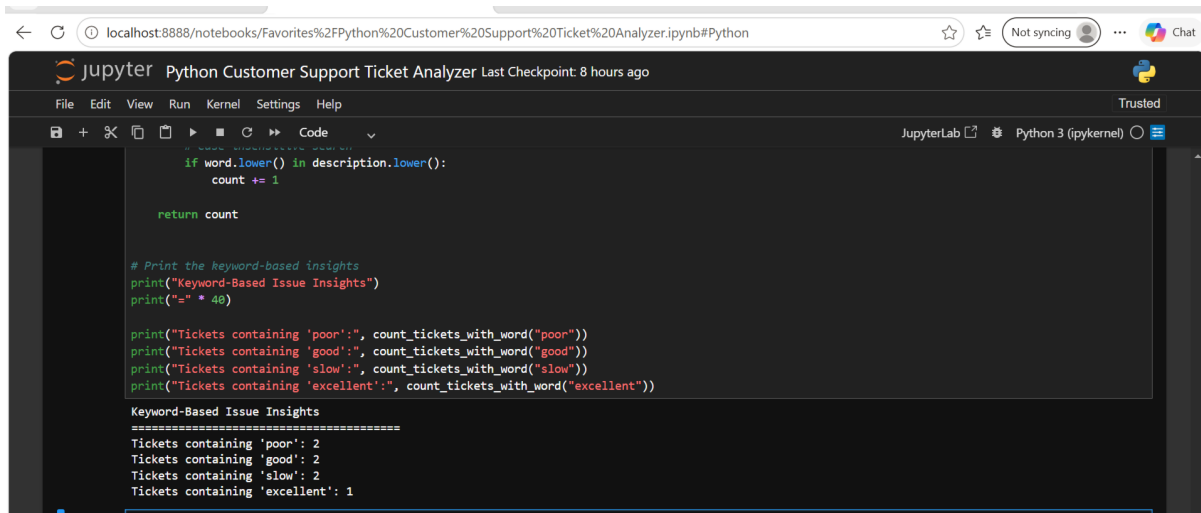
Function Requirements:

- Case-insensitive search
- Returns how many ticket descriptions contain the given word

Use this function to print:

- Number of tickets containing "poor"
- Number of tickets containing "good"
- Number of tickets containing "slow"
- Number of tickets containing "excellent"

OUTPUT:



```
localhost:8888/notebooks/Favorites%2FPython%20Customer%20Support%20Ticket%20Analyzer.ipynb#Python
Jupyter Python Customer Support Ticket Analyzer Last Checkpoint: 8 hours ago
File Edit View Run Kernel Settings Help
+ - X Copy Paste Run Cell Run All Code
Python 3 (ipykernel)

# Count tickets with specific keywords
if word.lower() in description.lower():
    count += 1

return count

# Print the keyword-based insights
print("Keyword-Based Issue Insights")
print("=" * 40)

print("Tickets containing 'poor':", count_tickets_with_word("poor"))
print("Tickets containing 'good':", count_tickets_with_word("good"))
print("Tickets containing 'slow':", count_tickets_with_word("slow"))
print("Tickets containing 'excellent':", count_tickets_with_word("excellent"))

Keyword-Based Issue Insights
=====
Tickets containing 'poor': 2
Tickets containing 'good': 2
Tickets containing 'slow': 2
Tickets containing 'excellent': 1
```

Step 5: Final Summary & Insights

Produce detailed analytics:

1. Display Final Cleaned ticket_data

Nicely formatted dictionary-of-lists output.

2. Priority Analysis

Compute:

- Number of High-priority tickets
- Number of Medium priority tickets
- Number of Low-Priority tickets

3. Find the Ticket With the Longest Issue Description

Based on word count, print:

- Ticket number
- Customer name
- Cleaned issue text
- Word count

4. Extract Unique Words Used

Generate a set of all unique words across all issue descriptions.

Show:

- Count of unique words
- The word list (sorted)

OUTPUT:

```
localhost:8888/notebooks/Favorites%2FPython%20Customer%20Support%20Ticket%20Analyzer.ipynb#Python
Jupyter Python Customer Support Ticket Analyzer Last Checkpoint: 9 hours ago
File Edit View Run Kernel Settings Help
print(sorted_words)

FINAL CLEANED TICKET DATA
=====
Ticket_No: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12]
Customer_Name: ['Ravi', 'Meera', 'Sam', 'Anu', 'Rakesh', 'Divya', 'Arjun', 'Kiran', 'Leela', 'Nisha', 'Jaya', '4']
Issue_Description: ['internet not working', 'slow response very poor service', 'great support issue resolved', 'okayay need help', 'not bad but slow',
'excellent guidance very helpful', 'good support and good behaviour', 'poor handling of technical issue', 'satisfied could be better', 'good service qu
ick response', 'payment issue', 'network issue']
Priority: ['High', 'Low', 'High', 'Medium', 'Low', 'High', 'Medium', 'High', 'Low', 'Medium', 'High', 'medium']

PRIORITY ANALYSIS
=====
High-priority tickets : 5
Medium-priority tickets : 3
Low-priority tickets : 3

TICKET WITH LONGEST ISSUE DESCRIPTION
=====
Ticket Number : 2
Customer Name : Meera
Issue : slow response very poor service
Word Count : 5

UNIQUE WORD ANALYSIS
=====
Number of unique words: 33
Sorted unique words:
['and', 'bad', 'be', 'behaviour', 'better', 'but', 'could', 'excellent', 'good', 'great', 'guidance', 'handling', 'help', 'helpful', 'internet', 'issu
e', 'need', 'network', 'not', 'of', 'okayay', 'payment', 'poor', 'quick', 'resolved', 'response', 'satisfied', 'service', 'slow', 'support', 'technica
l', 'very', 'working']
```

Conclusion:

The **Customer Support Ticket Analyzer** assignment provided practical experience in using Python to store, clean, analyze, and extract meaningful insights from customer support ticket data. The project involved creating preloaded ticket data, adding new tickets dynamically, validating ticket priorities, and cleaning issue descriptions by removing unnecessary spaces and punctuation, converting text to lowercase, and replacing common shorthand.

The project also demonstrated how Python functions can be used to perform **keyword-based analysis** to identify tickets containing words such as *poor*, *good*, *slow*, and *excellent*. The final analysis included priority distribution, identification of the ticket with the longest issue description, and extraction of unique words from all ticket descriptions.

Overall, this assignment strengthened important Python skills such as **lists, dictionaries, sets, loops, functions, string manipulation, conditional statements, and data analysis**. These skills can be applied to real-world customer service datasets to identify common issues, understand support quality, and support data-driven improvements in customer service operations.